

FinAccrual ERP Platform

Authentication and Authorization Details

How users, administrators and ERP connections are identified and controlled

Item	Detail
Project	FinAccrual ERP Platform (Backend API, Excel Add-in, Admin Dashboard)
Document	Authentication and Authorization Details

This document explains how sign-in works, how tokens are issued and refreshed, how routes are protected, and how roles, plans and ownership rules restrict what each caller can do.

1. Overview

FinAccrual uses three separate authentication mechanisms:

Mechanism	Who uses it	Method	Result
User sign-in	Accountants using the Excel add-in / dashboard	OAuth 2.0 with Google or Microsoft (Entra ID)	Application JWT (1 h) + opaque refresh token (7 days)
Administrator sign-in	Legacy admin module	E-mail + password (bcrypt) at /api/admin/login	Admin JWT (24 h) with role = admin
ERP connection	Backend acting for a user	OAuth 2.0 with QuickBooks Online and Xero	Provider access + refresh tokens stored per company

Note: Local e-mail/password sign-up for normal users is switched off. AuthService.signup() and login() always return the message "Local password authentication is disabled. Please sign in using Google or Microsoft OAuth." Only Google and Microsoft sign-in work for users.

2. User Authentication (Google and Microsoft)

2.1 Sign-in flow

1. The add-in opens a pop-up to GET /api/auth/google/connect or GET /api/auth/microsoft/connect (optional login_hint e-mail to skip the account chooser).
2. The backend redirects the pop-up to the provider's authorisation page (Google: openid email profile; Microsoft: openid profile email User.Read offline_access).
3. After consent the provider redirects to /api/auth/google/callback or /api/microsoft/callback with a code.
4. The backend exchanges the code for provider tokens (server-side, using the client secret) and reads the user profile (Google userinfo, Microsoft Graph /me).
5. The user is looked up by provider ID, then by e-mail. An existing user gets the provider ID attached; otherwise a new users row is created with role =

user and no plan.

6. A token pair is generated and stored on the user row (see section 3).
7. The pop-up page sends { type, email, name, token, refreshToken } to the task pane with postMessage (or Office.context.ui.messageParent inside an Office dialog) and closes.
8. The add-in verifies the message origin, saves the session in localStorage and shows the dashboard or the plan-selection screen.

2.2 Account rules

- ☐ E-mail addresses are lower-cased and trimmed; email is unique, so the same person cannot have two accounts.
- ☐ A Microsoft account without an e-mail or user principal name is rejected.
- ☐ New accounts have **no plan**. The user must choose **Start Free Trial** (POST /api/auth/start-trial) or a paid plan before the trial clock or plan limits apply.
- ☐ Setting users.is_active = 0 blocks the user on every protected route (the middleware reloads the user on each request).
- ☐ Provider errors are shown in a small error page in the pop-up and reported back as google_error / microsoft_error.

3. Tokens and Sessions

Token	Format	Lifetime	Stored where	Purpose
Access token	JWT (HS256) signed with JWT_SECRET. Payload: userId, email, role	1 hour	Client localStorage (fa_jwt_token); last value also in users.access_token	Authenticates API calls (Authorization: Bearer <jwt>)
Refresh token	Random 64 bytes as 128 hex characters (not a JWT)	7 days	Client localStorage (fa_refresh_token); users.refresh_token	Obtain a new token pair
Admin token	JWT signed with the same secret. Payload: id, email, role = admin	24 hours	Admin dashboard localStorage (token)	Admin endpoints
Server	express-session	24 hours	Server memory	OAuth state,

session	cookie, in-memory store			pending ERP connection data
---------	-------------------------	--	--	-----------------------------

3.1 Refresh and rotation

- POST /api/auth/refresh with { "refreshToken": "..." } looks up the user by that token, checks is_active and expiry, then issues a **new pair** and overwrites the old one (rotation). Only one refresh token per user is valid at a time.
- An unknown, reused or expired refresh token returns 401 ERR_INVALID_CREDENTIALS.
- Signing in on a second device replaces the first device's refresh token because only one is stored per user.

3.2 Logout

POST /api/auth/logout finds the user (from the JWT, refresh token or session), sets access_token, refresh_token and their expiry columns to NULL, and destroys the server session. It is deliberately not behind authenticate, so a client with an expired JWT can still log out. Note that an access JWT that has already been issued remains cryptographically valid until it expires; the server does not keep a deny-list.

3.3 How the add-in handles expiry

- Before each request apiFetch decodes the JWT locally; if exp has passed it raises the fa_session_expired event instead of calling the API.
- A 401 from the server also raises fa_session_expired, and the add-in returns to the sign-in screen. A 401 with code ERR_ERP_SESSION_EXPIRED raises fa_erp_session_expired instead (reconnect the ERP company).
- The refresh token is saved by the add-in, but the current front-end code does not call /api/auth/refresh; users sign in again after the 1-hour access token expires. Implementing a silent refresh in apiFetch would remove this step.

4. Request Authentication (Middleware)

4.1 authenticate (user routes)

File: `src/modules/auth/auth.middleware.js`. Used on almost every protected route.

1. Read the token from the Authorization: Bearer header. If missing, fall back to `?token=` in the query string (needed for pop-up GET requests such as ERP connect and checkout), then to token in the JSON body.
2. Verify the signature and expiry with `JwtService.verifyToken`.
3. Load the user by `userId`; reject if not found or `is_active` is false.
4. Set `req.user = { userId, email, role, name }` from the database (not from the token payload).
5. Any failure is forwarded as `SessionExpiredError` → 401 `ERR_SESSION_EXPIRED`.

4.2 authenticateJWT (admin routes)

File: `src/core/middleware/authMiddleware.js`. Verifies the Bearer token with the same secret and sets `req.user` to the decoded payload. It does not check the database, so it must be combined with `authorize`.

4.3 authorize(...roles)

File: `src/core/middleware/authorize.js`. Runs after an authentication middleware and compares `req.user.role` with the allowed roles. If the role is missing or not allowed it returns 403 `ERR_FORBIDDEN` ("You do not have permission to perform this action."). Because user and admin tokens share one secret, this check is what stops a normal user's token from working on admin endpoints.

```
router.get('/users', authenticateJWT, authorize('admin'),  
AdminController.listUsers);
```

5. Authorization Model

5.1 Roles

Role	Where stored	Capabilities
user	users.role (default)	Own profile, own plan, own ERP connections, own notifications, Excel validation, master-data pulls
admin	users.role = admin or an admins login	Everything a user can do plus GET /api/admin/me and GET /api/admin/users

5.2 Route protection matrix

Access level	Routes
Public	GET /api/health GET /api/auth/{google microsoft}/connect and /callback (and the /api/google/*, /api/microsoft/* aliases) POST /api/auth/refresh (rate-limited) POST /api/auth/logout POST /api/auth/signup and /login (always rejected – local auth disabled) GET /api/payments/checkout (validates the optional token if given) POST /api/admin/login, POST /api/admin/signup
Public but state-protected	GET /api/quickbooks/callback and GET /api/xero/callback require an OAuth state equal to the value saved in the caller's session. POST /api/xero/select-companies works only on the pending tokens held in the caller's server session.
Authenticated user (authenticate)	GET /api/auth/me, POST /api/auth/update-plan, POST /api/auth/start-trial POST /api/subscription/upgrade, POST /api/payments/complete /api/connections, /api/connections/stats, /api/connections/:id (delete, activate, count, rename) GET /api/pull-master-data /api/quickbooks/* and /api/xero/* data, connection and pull endpoints

	/api/notifications (list, create, mark-read, clear) /api/excel-validation/* (schema-check, data-type-check, report, vs-api, vs-database)
Admin only (authenticateJWT + authorize('admin'))	GET /api/admin/me GET /api/admin/users

5.3 Data ownership

Every ERP and notification query is scoped by the **verified** req.user.userId (never by a client-supplied e-mail or ID), so one user cannot list, rename, activate, disconnect or pull data for another user's companies or read another user's notifications. A request for a company the caller does not own behaves as "not found".

5.4 Subscription plans and limits

Plan	Connections per ERP	Notes
trial	1	Started by POST /api/auth/start-trial; length set by TRIAL_DURATION_MS (default 2 minutes for testing)
Basic	1	Paid plan
Standard	3	Paid plan
Pro	10	Paid plan; default when no plan is supplied to some endpoints

- ☐ The limit is checked **before** the OAuth redirect and again in the callback; exceeding it shows a "Connection Limit Reached" page.
- ☐ A downgrade (for example Pro → Basic) through the billing routes emits a user.downgraded event; the QuickBooks and Xero services listen for it and clear that user's connections. POST /api/auth/update-plan attempts the same clean-up directly, but its query uses a mail column that does not exist (see the database document).
- ☐ Billing plan names are exact-case (Basic, Standard, Pro); the update-plan schema uses lower-case names (trial, basic, standard, pro).

6. Administrator Authentication

Endpoint	Behaviour
POST /api/admin/signup	Validates name, email, password (min 6). Rejects a duplicate e-mail. Hashes the password with bcrypt (cost 10), stores the admin, returns an admin JWT (24 h).
POST /api/admin/login	Finds the admin by e-mail, compares with bcrypt.compare, returns the admin DTO (no password) and a JWT. The same message "Invalid email or password." is used for unknown e-mail and wrong password.
GET /api/admin/me	Returns the decoded token payload. Requires the admin role.
GET /api/admin/users	Lists all users (id, name, e-mail, provider, role, active flag, plan, created date). Requires the admin role.

6.1 Ways to obtain admin rights

- ☐ Create an account in the admins table through /api/admin/signup, or
- ☐ Promote an existing Google / Microsoft user in SQL:

```
UPDATE users SET role = 'admin' WHERE email = 'person@example.com';
```

The Admin Dashboard front end currently posts to /api/auth/login and /api/auth/signup, which are disabled for local authentication, so administrators should use the Google sign-in button on that page (which requires the user to have role = admin) or the front end should be pointed at /api/admin/login.

7. ERP (QuickBooks and Xero) Authorization

7.1 Connecting a company

1. The add-in opens GET
/api/{quickbooks|xero}/connect?tier=...&token=<jwt> in a pop-up.
authenticate identifies the user from the token; oauthLimiter and Joi validate the request.

2. The backend checks plan limits (or, for *Reconnect*, that the company belongs to the user).
3. A random state (16 bytes, base64url) is created and saved in the server session with the user ID, tier and reconnect target.
4. The user is redirected to Intuit or Xero to approve access.
5. On callback, `validateQuickBooksState` / `validateXeroState` require the returned state to equal the session value. A missing session or mismatch is rejected (CSRF protection).
6. The code is exchanged for tokens on the server. QuickBooks stores the company immediately; Xero first shows a company-selection page and saves the chosen tenants via `POST /api/xero/select-companies`.
7. A reconnect is accepted only if the authorised company matches the one being reconnected.

7.2 Keeping ERP tokens valid

- ☐ `TokenManager` (shared by both providers) returns the stored access token if it has more than one minute left; otherwise it refreshes it.
- ☐ A per-company lock (`LocalLockManager`) ensures only one refresh runs at a time; other requests wait and reuse the new token.
- ☐ If the refresh token itself has expired, or the provider answers `invalid_grant` / `invalid_token` / `unauthorized`, the connection is marked `Disconnected` and its tokens are wiped. The API answers `401 ERR_ERP_SESSION_EXPIRED` and the user must reconnect.
- ☐ Network or provider outages during refresh return `502 ERR_ERP_REFRESH_FAILED` without disconnecting the company.

8. Security Controls

Control	Details
Password hashing	bcrypt, cost factor 10 (admin accounts only)
Rate limiting	Production: 300 req/min general; 20 req/15 min on login, signup and refresh; 50 req/15 min on ERP connect/callback. Exceeding returns <code>429 ERR_LIMIT_REACHED</code> . Disabled in development
Security headers	Helmet (CSP and COOP disabled so OAuth pop-ups work)

Input validation	Joi schemas per route (types, enums, lengths); unknown fields stripped where configured
Sanitisation	Strips control characters, <script> blocks and inline event handlers from request bodies and params
Content-Type check	JSON bodies must use application/json
OAuth CSRF	Random state per connect, verified against the server session
Origin check (client)	The add-in accepts postMessage data only from its own origin, the backend origin, or localhost in non-production builds
Model validation	Sequelize validators for e-mail format, role, status, notification type
Centralised errors	All failures use one envelope; raw stack traces and SQL errors are never sent to clients

8.1 Error response format and codes

```
{ "success": false, "code": "ERR_SESSION_EXPIRED", "message": "Your session has expired. Please sign in again.", "details": "..." }
```

Code	HTTP	Meaning
ERR_SESSION_EXPIRED	401	JWT missing, invalid, expired, or user inactive
ERR_INVALID_CREDENTIALS	401	Bad credentials, disabled local login, or invalid refresh token
ERR_ERP_SESSION_EXPIRED	401	QuickBooks / Xero connection must be re-authorised
ERR_FORBIDDEN	403	Authenticated but role not allowed
ERR_VALIDATION	400	Invalid input, bad OAuth state, failed model validation
ERR_NOT_FOUND	404	Unknown route or resource not owned by the caller
ERR_LIMIT_REACHED	429	Rate limit or plan limit reached
ERR_ERP_REFRESH_FAILED	502	Temporary failure refreshing an ERP token
ERR_CONNECTION_REFUSED	503	Database or upstream service unreachable
ERR_INTERNAL	500	Unexpected server error

9. Hardening Recommendations

The following points were identified while documenting the code. They are grouped by priority so the team can plan fixes before a public release.

Priority	Finding	Recommendation
High	POST /api/admin/signup is public. Anyone who can reach the API can create an admin account and receive an admin token. The rate limiter is off outside production.	Remove the route, or require an existing admin token / a one-time setup key.
High	POST /api/payments/complete takes email from the request body and upgrades that user's plan; the caller only needs to be logged in, and no real payment is verified (mock checkout).	Use req.user instead of a body e-mail and confirm payment with the payment provider's webhook before changing a plan.
High	The session secret is a fixed string in code, and JWT_SECRET has a public default value.	Read both from the environment and refuse to start in production without them.
High	ERP and refresh tokens are stored in plain text in MySQL.	Encrypt sensitive columns at rest; hash refresh tokens before storing.
Medium	The JWT is accepted from the query string and request body, which can leak into logs and browser history.	Limit query-string tokens to the pop-up routes that need them.
Medium	The pop-up pages insert name, email and tokens into inline JavaScript and call <code>postMessage(..., "*")</code> .	Escape values (for example with <code>JSON.stringify</code>) and target the exact add-in origin.
Medium	CORS uses <code>origin: true</code> with credentials.	Allow only the add-in, admin and API origins.
Medium	The front end does not use the refresh token, so users must sign in every hour; comments and README mention a 15-minute token although the code uses 1 hour.	Implement silent refresh; update comments and README.
Medium	Sessions and refresh locks are in memory, so they do not work across several instances or after restart.	Use a shared session store and a distributed lock when scaling out.
Low	The Admin Dashboard posts to disabled endpoints and to /api/launch-excel, which does not exist in the backend routes.	Point the dashboard at /api/admin/* and remove or implement the missing route.
Low	update-plan and trial_ends_at issues	Fix the column names and add

	described in the database document.	the missing model field.
--	-------------------------------------	--------------------------